IEEE 754 Floating-Point Representation and Arithmetic

Week 3 — Real Numbers in a Finite Number of Bits

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 7, 2026

Where We Left Off (Week 2)

Week 2 was about *integers and their hardware*

- Number systems and conversion.
- Two's complement: negatives as ordinary bits, range -2^{n-1} to $2^{n-1}-1$.
- Adders: HA $\rightarrow$ FA $\rightarrow$ ripple-carry; $A - B = A + (-B)$ free.

The pivot today

Integers cover a small, fixed range. Real numbers in science span 10^{-300} to 10^{300} — a single fixed point cannot cover them. This week: the **IEEE 754** floating-point representation — sign, exponent, mantissa — that trades precision for range.

Roadmap: Four Questions, One Thread

1. **Scientific notation** — the idea floating point is built on.
2. **IEEE 754 single precision** — the 32-bit format, bit by bit.
3. **Normalization, bias, special values** — how the format stays efficient and correct.
4. **Floating-point arithmetic** — how add/multiply actually run, and where precision is lost.

Running thread for today

We will encode one number — **10.75** — in IEEE 754 single precision, then add it to another and see where precision gets lost.

Floating point is scientific notation, compressed into 32 bits.

Motivation: One Sign, One Mantissa, One Exponent

Socratic question

Write $0.000\,000\,000\,000\,000\,000\,000\,000\,000\,001$ in science notation. How many digits of the value does the "1" carry — and how many bits would a fixed point waste on leading zeros?

- Scientific notation: 1.0×10^{-30} — three fields, huge range, constant precision.
- **Sign** ($\pm$), **mantissa/significand** (the digits), **exponent** (where the point goes).
- A fixed point would need hundreds of bits for this value; floating point needs the mantissa's precision only.

Definition: Floating-Point Value

General form

A floating-point number represents

$$(-1)^S \times M \times r^E,$$

where S is the **sign** bit, M the **mantissa**, E the **exponent**, and r the base (2 in hardware).

- Range comes from the exponent; precision from the mantissa — independent knobs.
- Base 2, not 10: binary scientific notation $1.xxx_2 \times 2^E$.
- This is the format IEEE 754 standardizes for base 2.

(P&H, Sec. 3.5, p.205; Hamacher Ch. 9) [2, 1]

Worked Example: Binary Scientific Notation

Write 10.75 in binary scientific notation

$10.75 = 8 + 2 + 0.5 + 0.25 = 1010.11_2$.

Normalize

$1010.11_2 = 1.01011_2 \times 2^3$. So: sign $S = 0$, mantissa $= 01011$, exponent $E = 3$.

This is the running thread's starting point — we will encode it in IEEE 754 single precision next.

Socratic Check: What Does Normalization Buy?

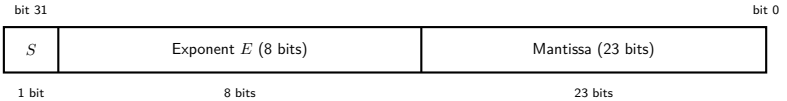
Question

Any binary number has many representations ($10.75 = 0.101011 \times 2^4 = 1.01011 \times 2^3$). If we agree the mantissa always starts with a leading 1 ("normalized"), what do we gain?

- A **unique** representation — every value has exactly one form.
- We can **omit the leading 1** (it is implicit) — one extra bit of precision for free.
- This is exactly what IEEE 754 does: a hidden "1." in front of the stored mantissa.

Scientific notation is the idea. Now: the 32-bit format that standardizes it.

The 32-Bit Layout



- **Sign** S: 1 bit. **Exponent** E: 8 bits. **Mantissa** M: 23 bits (fraction only; leading 1 hidden).
- Value $= (-1)^S \times (1.M)_2 \times 2^{E-\text{bias}}$.
- **Bias** $= 127$ for single precision: the stored 8-bit exponent is $E_{\text{stored}} = E_{\text{true}} + 127$.

(P&H, Sec. 3.5, p.205) [2]

Why Bias the Exponent?

The problem

The true exponent can be negative ($10.75 = 1.01011 \times 2^3$ has $E = 3$, but $0.5 = 1.0 \times 2^{-1}$ has $E = -1$). How do we store a signed exponent without two's complement complicating comparison?

- **Bias**: store $E_{\text{stored}} = E_{\text{true}} + 127$, an unsigned number.
- Result: the **all-zeros** exponent means "smallest," the all-ones means "largest" — unsigned comparison orders floats.
- $E_{\text{true}} = -126$ to $+127$ (single); the extreme all-0 and all-1 exponents are reserved (special values, next act).

Example

10.75 has $E_{\text{true}} = 3 \Rightarrow E_{\text{stored}} = 3 + 127 = 130 = 1000\,0010_2$.

Worked Example: Encoding 10.75

Assemble the three fields

From Act 1: $10.75 = 1010.11_2 = 1.01011_2 \times 2^3$, so $S = 0$, mantissa $M = 01011\,0000\,0000\,0000\,0000\,000$ (23 bits), exponent $E_{\text{stored}} = 130 = 1000\,0010_2$.

The full 32 bits

$$\underbrace{0}_{\text{sign}}\ \underbrace{1000\,0010}_{\text{exp=130}}\ \underbrace{01011000000000000000000}_{\text{mantissa } 01011\ldots}$$

`0x412C0000` (hex).

Check: $1.01011_2 \times 2^{130-127} = 1.01011_2 \times 2^3 = 1010.11_2 = 10.75$. Correct.

Socratic Check: Decoding Back

Question

Decode `0x3F800000` (sign 0, exponent $0111\,1111_2 = 127$, mantissa all zeros). What value is it — and what does that tell you about the most "normal" number in the format?

- $E_{\text{true}} = 127 - 127 = 0$; mantissa 0; value $= 1.0 \times 2^0 =$ **1.0**.
- The bias is chosen so that **1.0** is the most ordinary pattern (exponent $0111\,1111$).
- This is the "anchor" number — check your encoding by seeing how far the exponent field is from 127.

The format is fixed. Now the edges: the special values and the arithmetic.

Special Values: When the Format Runs Out

Reserved exponent patterns

- **Zero**: exponent all 0, mantissa all 0 ($+0$ and -0).
- **Denormal (subnormal)**: exponent all 0, mantissa nonzero — numbers smaller than the smallest normalized; leading 1 is *not* assumed.
- **Infinity**: exponent all 1, mantissa all 0 ($\pm\infty$).
- **NaN** (Not a Number): exponent all 1, mantissa nonzero — $0/0$, $\sqrt{-1}$.

These four patterns give the format a well-defined behavior at its extremes instead of undefined results. (P&H, Sec. 3.5, p.205) [2]

Worked Example: The Smallest and Largest

Single precision, normalized range

- Smallest normalized: exponent 1, mantissa 0 $\Rightarrow 1.0 \times 2^{-126} \approx 1.18 \times 10^{-38}$.
- Largest normalized: exponent 254, mantissa all 1
  $\Rightarrow (2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$.

The point

Roughly 10^{-38} to 10^{38} — compare with fixed point's tiny range. But note: only about 7 **significant decimal digits** (precision is fixed by the 24-bit mantissa including the hidden 1).

Rounding and Precision Loss

The hard truth

Most real numbers are *not* exactly representable — 0.1 repeats in binary. IEEE 754 rounds to the nearest representable value (with ties to even).

- **Precision**: 24 bits of mantissa $\approx$ 7 decimal digits.
- **Rounding**: the stored mantissa is the closest fit; error $\leq \frac{1}{2}$ ULP (unit in the last place).
- Consequence: **floating-point arithmetic is approximate**; equality tests on floats are dangerous.

(P&H, Sec. 3.5, p.205) [2]

Worked Example: Where Precision Is Lost

The thread continues

Add $10.75 + 0.1$ in single precision. 0.1 is stored as an approximation: $0.1 \approx 1.10011001100110011001101_2 \times 2^{-4}$ (rounded to 24 bits).

The catch

When you add a *large* and a *small* number, the small one's low bits are pushed out of the 24-bit window — $10.75 + 0.1$ in single precision may round to 10.85 exactly, but adding $1\,000\,000 + 0.1$ rounds to $1\,000\,000$ (the 0.1 vanishes entirely).

Takeaway

Precision is lost at the **least significant** digits, and the loss is worst when adding numbers of very different magnitudes.

Socratic Check: Why Not Compare Floats for Equality?

Question

In a loop you compute $x = 0.1$ ten times by addition. Is $x == 1.0$ guaranteed true? What happens, and why?

- No — each 0.1 is an approximation; ten additions accumulate rounding error, so x is 1.0000000000000002-ish, not exactly 1.0.
- Rule: compare floats with a **tolerance** ($|a - b| < \epsilon$), never with $==$.
- This is the practical consequence of finite precision.

The format and its edges are set. Now: how does the hardware actually add two floats?

Motivation: Adding Two Exponents

Socratic question

To add $1.5 \times 2^3 + 1.25 \times 2^5$, you cannot just add the mantissas — their exponents differ. What is the first thing the hardware must do?

- **Align exponents**: shift the smaller mantissa right until both exponents match.
- Then add the mantissas, re-normalize, and round.
- The alignment is exactly where the low-bit loss of the last act comes from.

The Floating-Point Add Algorithm

Four steps (P&H, Sec. 3.5, p.205) [ ]

1. **Align**: shift the mantissa of the smaller-exponent operand right until exponents are equal (track the shifted-out bits).
2. **Add**: add the (aligned) mantissas.
3. **Normalize**: shift the result to have a leading 1; adjust the exponent.
4. **Round**: round the mantissa to the stored width (ties to even).

Multiply is easier: multiply mantissas, add exponents (subtract bias once), normalize, round — no alignment step.

Worked Example: Adding 10.75 + 0.1 (Qualitative)

Align, add, normalize, round

$10.75 = 1.01011_2 \times 2^3$; $0.1 \approx 1.100110\ldots_2 \times 2^{-4}$.

The trace

- Align: shift 0.1's mantissa right by $3 - (-4) = 7$ places.
- Add mantissas, re-normalize, round to 24 bits.
- Result ≈ 10.85 — but the alignment **truncated** low bits of 0.1, so the sum is slightly off from the exact value.

Notice

The error is invisible in the 7-digit window; it would only show if we subtracted back or accumulated many such adds. Precision is *silent* loss.

Socratic Check: Multiplication, One Line

Question

What are the three fields of the *product* of two normalized floats, in terms of the inputs' fields — and why is the exponent easier than in addition?

- Sign $S = S_1 \oplus S_2$; mantissa $M = M_1 \times M_2$ (re-normalized); exponent $E = E_1 + E_2 - \text{bias}$.
- No alignment needed (exponents just add) — but the mantissa product is wider, needing normalization and rounding.
- Symmetric to addition: multiply mantissas, add exponents, normalize, round.

One format, four steps of arithmetic, and a silent precision budget.
Now: put it together.

The Thread, Closed

What we built today

- **Scientific notation** $\rightarrow$ sign, mantissa, exponent.
- **IEEE 754 single**: 1 sign + 8 exponent + 23 mantissa, bias 127, hidden leading 1.
- **Special values**: zero, denormal, infinity, NaN; reserved exponent patterns.
- **Arithmetic**: add = align, add, normalize, round; multiply = multiply mantissas, add exponents, normalize, round.
- **Precision**: 7 decimal digits; rounding is silently approximate.

The running thread

$10.75 = 1.01011_2 \times 2^3$ encoded as `0x412C0000`; adding 0.1 showed alignment truncating low bits — precision loss is silent.

Bridge to Week 4

What's still missing

We now know how numbers are represented (integers, fixed point, IEEE 754) and how the ALU computes on them. But bits and adders do not run a program by themselves.

Next week

The machine: registers (the fast scratchpad), the stack (LIFO operand home), instruction formats, and the fetch–decode–execute cycle that animates them — the storage spaces and the cycle.

Summary

- **Floating point** $= (-1)^S \times 1.M \times 2^{E-\text{bias}}$: range from the exponent, precision from the mantissa.
- **Single precision**: 32 bits $= 1 + 8 + 23$, bias 127, hidden 1.
- **Special values**: zero/denormal/infinity/NaN from reserved exponent patterns.
- **Add**: align, add, normalize, round. **Multiply**: multiply mantissas, add exponents, normalize, round.
- **Precision**: 7 digits; rounding is silent and approximate — never compare floats with $==$.

References

V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.
Computer Organization.
McGraw-Hill, 5th edition, 2002.

David A. Patterson and John L. Hennessy.
Computer Organization and Design: The Hardware/Software Interface.
Morgan Kaufmann, arm edition (5th) edition, 2017.